Paradigmas de Programação — Aula 09: Modelo de atores com Elixir

Elixir

A BEAM e o processo

- **Elixir**: funcional, tipagem dinâmica, criada por José Valim (2012); executa sobre a **BEAM**, a máquina virtual de Erlang
  - Erlang: criada na Ericsson para telefonia que não podia parar (ARMSTRONG, 2003) — aqui, só como origem
  - Elixir compila para o mesmo *bytecode* e usa a mesma biblioteca de processos, o **OTP**
- **Processo da BEAM**: pilha, *heap* e caixa de correio próprios, sem memória compartilhada — um **ator**

	Processo do SO	**Thread do SO**	**Processo da BEAM**
Escalonado por	SO	SO	máquina virtual (*M:N*, um escalonador por núcleo)
Memória	isolada	compartilhada	isolada
Comunicação	*pipe*, *socket*	variável compartilhada	mensagem
Custo	novo espaço de endereçamento	pilha ~1 MB	poucos kilobytes; centenas de milhares por programa

Átomos, tuplas e *pattern matching*

```elixir
:ok == :ok                          # átomo: o valor é o próprio nome -> true

def dividir(_dividendo, 0), do: {:erro, :divisao_por_zero}
def dividir(dividendo, divisor), do: {:ok, dividendo / divisor}  # tupla {:ok, valor} / {:erro, motivo}

{:ok, resultado} = dividir(10, 4)   # = é casamento: resultado = 2.5
[primeiro | resto] = [1, 2, 3]      # primeiro = 1, resto = [2, 3]

case divisao do
  {:ok, valor} -> "resultado #{valor}"     # #{...}: interpolação
  {:erro, motivo} -> "falhou: #{motivo}"
end
```

- A falha volta como **valor**, como o `(resultado, error)` de Go
- Valor que não casa com o padrão do `=`: erro de execução

Cláusulas, recursão e `|>`

```elixir
defmodule Basico do
  def classificar(numero) when numero < 0, do: :negativo   # guarda
  def classificar(0), do: :zero
  def classificar(_numero), do: :positivo             # _ casa com tudo

  def somar([]), do: 0                              # caso base
  def somar([cabeca | cauda]), do: cabeca + somar(cauda)

  def pares_ao_quadrado(lista) do
    lista
    |> Enum.filter(fn numero -> rem(numero, 2) == 0 end)   # x |> f(a) == f(x, a)
    |> Enum.map(fn numero -> numero * numero end)
  end
end
```

- Vale a **primeira cláusula** que casa; função identificada por nome e aridade: `classificar/1`
- Recursão no lugar de laço — `somar` **não** é de cauda: a soma ocorre depois do retorno

Maps, imutabilidade e a saída

```
pagamento = %{id: "pg-1", valor: 120.0}
aprovado = Map.put(pagamento, :status, :aprovado)   # devolve um map NOVO
aprovado.valor                                      # 120.0
```

Seção de `01-basico.exs`	Saída
Tuplas e *pattern matching*	`10 / 4 = 2.5`, `primeiro: 1, resto: [2, 3]`, `resultado 3.0`, `falhou: divisao_por_zero`
Cláusulas e guarda	`[:negativo, :zero, :positivo]`
Recursão e `|>`	`soma: 10`, `[4, 16, 36]`
Maps	`%{id: "pg-1", valor: 120.0}` e `%{id: "pg-1", status: :aprovado, valor: 120.0}`

- Todo dado é imutável (aula 07): o `pagamento` original continua sem `:status`
- Consequência para atores: o estado muda só porque a função é **chamada de novo** com outro argumento — a terceira ação do ator